certainly not production ready. For example, the `Log` method is not asynchronous, but you are calling an asynchronous API within it—hence the use of the `Task.Run` method. The best practice for writing logs to email, databases, or other *slow* (asynchronous) log stores is writing to an intermediate store, such as a buffer or message queue and then processing the queue separately. (See http://mng.bz/XaAa for more guidance.) In more complex applications, the general advice is to look for a third-party logger that offers all the features you need rather than trying to roll your own.

12.3.8 Structured logging with Serilog

Having explored the desirability of using a proven, tested, third-party logging solution, in this section, we will look at the steps required to implement logging with Serilog (https://github.com/serilog) in a Razor Pages application. The default implementation of Serilog includes logging providers (or *sinks*, in Serilog parlance) for console, file, and debug. The file option writes to a *rolling* log file, which means you can configure the logging system to create a new log file for each day, each hour, or after the file has reached a certain size, for example. In addition, you can specify that the file output is formatted as JSON, so you can see the structure of the logs, which will hopefully give you some idea of the usefulness of structured logging.

As well as the default sinks, there is a massive ecosystem of alternative sinks (http://mng.bz/ya2d) that target a wide variety of outputs, including email, databases, message queues, cloud-based service providers, and more. Just like .NET Core logging, Serilog outputs to all configured sinks. In this walk-through, you will configure Serilog to output to the console and a file as JSON.